ARP Éducation

Documentation fonctionnelle et technique

Application web PWA — Next.js + MongoDB — gestion de scolarité

Projet — cahier des charges ARP éducation

Version 3.1 — 7 novembre 2026

Remplace la version 2.0 du 19 octobre 2026 : base MongoDB, module Cours/RAG/QCM, assistant IA, anti-robot et pièces jointes n'y figuraient pas.

Sommaire

1. Présentation du projet
2. Besoins fonctionnels
3. Besoins non fonctionnels
4. Matrice des autorisations
5. Modèle de données MongoDB
6. Règles de gestion et cycles de vie
7. Architecture technique
8. Interface HTTP (pages et API)
9. Sécurité et anti-robot
10. Module Cours : documents, RAG et QCM noté /20
11. Assistant : Gemini avec repli local
12. PWA : installation et mode hors ligne
13. Installation, démarrage, démonstration
14. Préparer un déploiement hébergé
15. Tests réalisés et résultats
16. Limites connues et évolutions possibles
17. Annexe — arborescence

1. Présentation du projet

ARP Éducation est une application web de gestion de scolarité couvrant les six domaines du cahier des charges — utilisateurs, présences et emplois du temps, matières/salles/notes, comptes rendus, réclamations et notifications — et étendue par trois modules ajoutés en cours de projet : **Cours** (supports, recherche sémantique et QCM auto-corrigé), **assistant** conversationnel en lecture seule, et **mode hors ligne** de l'emploi du temps pour toute l'année scolaire. Tout est développé dans un seul projet Next.js qui assure à la fois le front et le back, adossé à une base **MongoDB** locale, et se présente comme une application installable (PWA) utilisable sur PC et sur téléphone, en ligne comme hors ligne.

Le produit compte 18 écrans, 22 points d'entrée d'API, 18 modules métier côté serveur (`lib/`), 14 composants d'interface et 17 collections. Trois rôles coexistent : `admin`, `prof`, `etudiant`. Les écritures passent exclusivement par des formulaires HTML POST suivis d'une redirection 303 : aucun JavaScript client n'est nécessaire pour modifier la donnée, ce qui rend chaque écran testable en curl et garde un rendu serveur intégral.

1.1 Choix structurants

Exigence du cahier des charges	Décision réelle	Motivation
Base de données intégrée H2	SQLite d'abord (v1), puis **MongoDB Community 8.0.10** en local (v3)	H2 n'existe qu'en Java : aucun binding Node.js exploitable. SQLite était l'équivalent fonctionnel sans serveur ; MongoDB a été adopté pour se rapprocher d'un déploiement réel (documents, binaire, index, Atlas hébergé en option). Le modèle relationnel a été conservé sous forme de collections à identifiants numériques, ce qui a permis une migration scriptée sans réécrire les écrans.
Next.js récent	Next 13.5.11 (App Router)	La machine cible reste en Node 18.13, refus de monter de version ; les branches récentes de Next exigent Node 18.17+. Conséquence assumée : pas d'actions serveur, remplacées par des route handlers, choix homogène dans tout le projet.
Installable comme YouTube, PC et téléphone	Manifest et service worker écrits à la main	Répond à l'exigence sans framework PWA externe et permet une stratégie de cache entièrement documentée.
Fonctionner avec ou sans connexion	Capture automatique de l'emploi du temps de **toute l'année scolaire**, filtrée par rôle	Un bouton « Préparer le hors ligne » s'oubliait ; la sauvegarde est déclenchée à chaque navigation connectée, sans clic, et effacée à la déconnexion.
Contexte sécurisé pour le téléphone	Serveur HTTPS personnalisé sur le port **443** + certificat mkcert auto-régénéré	Chrome n'autorise l'installation PWA et les service workers qu'en HTTPS. Le port fixe évite de dicter un numéro de port à chaque changement d'adresse ; le certificat suit les adresses du poste toutes les 30 secondes, sans redémarrage.
Aide à l'élève	Assistant conversationnel **optionnel** : Gemini si une clé existe, moteur à règles sinon	Aucune fonctionnalité ne doit dépendre d'un service payant. Sans clé, l'assistant répond localement et le reste de l'application est inchangé.

1.2 Synthèse technologique

Couche	Technologie	Version installée
Framework full-stack	Next.js (App Router, React Server Components)	13.5.11
Vue / runtime	React et React DOM	18.3.1
Langage	TypeScript (strict)	5.9.3

Base de données	MongoDB Community (driver officiel `mongodb`)	serveur 8.0.10 / driver 7.7.0
Accès aux documents	Couche maison `lib/db.ts` : ids numériques, hydratation, index, jeu de démonstration	—
Style	Tailwind CSS (utilitaires + classes de composants maison)	3.4.19
Authentification	Implémentation maison : cookie HMAC-SHA256 + hachage scrypt	—
Anti-robot	reCAPTCHA **v2** à case à cocher, activé seulement si les deux clés existent	—
Intelligence artificielle	API REST Gemini : contenu + plongements, repli local intégral	liste de modèles en section 11
PDF	Générateur maison `lib/pdf.ts`, aucune dépendance	—
Serveur	`server-https.js` : Node `https` devant Next, port 443, certificat auto-adaptatif	Node 18.13
Dépendance résiduelle	`better-sqlite3`, conservée pour le seul script de migration	9.6.0

2. Besoins fonctionnels

Le tableau reprend besoin par besoin l'expression du cahier des charges, l'acteur réellement habilité après la refonte des droits, et l'état de réalisation. Les besoins BF-20, BF-21, BF-25 et BF-27 ont vu leur rédaction ajustée : la refonte de septembre-octobre a déplacé certaines écritures du rôle administrateur vers le professeur ou vers l'auteur du document.

ID	Besoin	Acteur	Priorité	Réalisation
BF-01	Un compte administrateur unique, fixe, pré-existant	Système	Obligatoire	Compte `admin@arp.edu` présent au démarrage ; la ligne est marquée « protégé » dans l'écran Utilisateurs et aucune action n'est proposée dessus
BF-02	Créer les comptes professeur et étudiant à partir de leur adresse électronique	Admin	Obligatoire	Module Utilisateurs ; le mot de passe n'est pas saisi, le champ reste vide jusqu'à l'activation
BF-03	Envoyer un courriel demandant la définition du mot de passe	Système	Obligatoire	Invitation consignée dans la boîte d'envoi (module Emails) avec lien `/reset?token=…`
BF-04	Le destinataire définit son mot de passe, puis se connecte	Prof / Étudiant	Obligatoire	Page publique `/reset` ; jeton à usage unique ; connexion bloquée tant que le mot de passe n'est pas défini
BF-05	Renvoyer l'invitation si nécessaire	Admin	Souhaité	Bouton « Renvoyer l'email », qui bascule sur « Nouveau mot de passe » dès que le compte est activé
BF-06	Définir les salles	Admin	Obligatoire	Module Salles : nom, bâtiment, capacité, type (salle, amphi, laboratoire), usage réel par créneaux
BF-07	Créer chaque année des classes à partir des comptes étudiants	Admin	Obligatoire	Module Classes : nom, filière, année scolaire ; vues grille et liste
BF-08	Sélectionner les étudiants dans une classe, à tout moment	Admin	Obligatoire	Interface à cases à cocher ; le retrait de l'ancienne classe et l'affectation se font dans la même écriture
BF-09	Créer les matières avec coefficient, professeur, salles et volume horaire	Admin	Obligatoire	Module Matières : code, nom, coefficient, heures, classe, professeur responsable
BF-10	Notifier par courriel après création d'une matière	Système	Obligatoire	Courriel d'affectation au professeur à la création comme au réaffectation, avec lien vers son module Notes
BF-11	Chaque semaine, définir le programme de la classe	Admin	Obligatoire	Éditeur hebdomadaire en popup : classe + lundi de semaine, deux créneaux par jour
BF-12	Maximum deux matières par jour, trois heures chacune	Règle	Obligatoire	Créneaux imposés 09:00–12:00 et 13:00–16:00 ; index unique (classe, semaine, jour, créneau) côté MongoDB
BF-13	Récupérer l'emploi du temps en PDF	Tous	Obligatoire	Bouton « Télécharger en PDF » : `lib/pdf.ts` produit un A4 paysage côté serveur, renvoyé en pièce jointe ; un bouton « Imprimer » reste disponible
BF-14	Le professeur consulte ses affectations	Prof	Obligatoire	Matières, Notes, Présences, Comptes rendus, Cours et Emplois filtrés sur les matières dont il a la charge
BF-15	Signaler les absences dans l'application	Prof	Obligatoire	Appel par séance (présent / absent / retard) sur la date choisie, upsert par étudiant
BF-16	Créer un compte rendu avec une durée de validité	Prof	Obligatoire	Module Comptes rendus : titre, consigne, matière, date limite. L'administrateur consulte, il ne crée plus
BF-17	Après expiration, vérifier les dépôts et attribuer une note	Prof	Obligatoire	Dépôts lisibles et notables (0–20 + commentaire) par le professeur responsable

BF-18	À la fin du cours, soumettre la clôture avec les notes d'examen final	Prof	Obligatoire	Formulaire de clôture listant tous les étudiants de la classe ; passe la matière au statut « terminée »
BF-19	Consulter ses absences	Étudiant	Obligatoire	Module Présences : compteurs, taux de présence, historique daté avec nom du relevé
BF-20	Consulter ses notes par matière et le total	Étudiant	Obligatoire	Module Notes : détail CC/TP/Examen, moyenne par matière, moyenne générale pondérée par coefficient sur les matières validées
BF-21	Déposer son compte rendu	Étudiant	Obligatoire	Dépôt jusqu'à la date limite, une fois par étudiant et par sujet (index unique). Le texte peut être saisi, repris d'une pièce jointe .docx/.pdf, puis modifié
BF-22	Télécharger son emploi du temps	Étudiant	Obligatoire	Vue forcée sur sa classe (une requête visant une autre classe est ignorée), téléchargement du PDF, mémorisation hors ligne
BF-23	L'administration valide la matière clôturée avant publication	Admin	Obligatoire	Bouton « Valider et publier aux étudiants », seulement depuis « terminée » ; les notes restent invisibles avant ce passage
BF-24	Seules les notifications créées par l'admin sont émises ; visibles par tous	Admin	Obligatoire	Formulaire réservé à l'admin ; liste globale sans filtre de destination
BF-25	Seuls les professeurs et étudiants créent des réclamations	Prof / Étudiant	Obligatoire	Création refusée à l'admin ; le *statut* est piloté par l'auteur seul (l'admin consulte), suppression par l'auteur tant que non traitée
BF-26	Terminer la semaine précédente comme base de travail	Admin	Complément	Bouton « Copier la semaine précédente »
BF-27	Afficher un tableau de bord adapté au rôle	Tous	Complément	Indicateurs différents par rôle, calendrier de la semaine, dernières notifications, accès direct au hors ligne

2.1 Besoins implicites ajoutés (non écrits dans le cahier des charges)

- Désactivation d'un compte plutôt que suppression, pour conserver l'historique de notes et de présences.
- Suppression protégée : une matière, une salle, une classe ou un compte rendu n'est supprimable que si rien ne le référence ; le refus s'accompagne d'un message explicite renvoyé vers la page.
- Validation serveur systématique : bornes de notes 0–20, types de pièce et de salle contraints, dates obligatoires, formats de fichiers limités.
- Double niveau de contrôle d'accès : chaque route rejette un rôle non autorisé par un 403, indépendamment de ce que masque l'interface.
- Jeu de démonstration recréé automatiquement dès que la collection `users` est vide.
- Filtres portés par l'URL : toute liste se filtre en GET, donc une URL est partageable, le bouton retour fonctionne et aucune requête réseau client n'est nécessaire.
- Obligation de choisir une classe avant d'afficher des données de masse, pour ne pas mélanger plusieurs promotions.

2.2 Fonctionnalités hors cahier des charges, ajoutées en cours de projet

ID	Fonctionnalité	Acteur	Description
AJ-01	Supports de cours	Prof	Chapitres numérotés 1–30 par matière, importés en .docx ou .pdf, avec extraction automatique du texte ; fermeture et réouverture d'un chapitre
AJ-02	Recherche sémantique (RAG)	Système	Découpage du cours en passages, plongements vectoriels Gemini stockés en base, similarité cosinus pour retrouver les passages pertinents

AJ-03	QCM auto-corrigé noté /20	Étudiant	6 questions par chapitre, 8 pour le QCM global, une seule tentative, correction immédiate, copie consultable
AJ-04	Tableaux de bord quiz	Prof / Étudiant	Copies, moyennes et taux de réussite par chapitre et par matière ; l'élève ne voit que la sienne
AJ-05	Assistant conversationnel	Tous	Bulle flottante, réponses en lecture seule, restreintes au compte, 5 familles de refus, quota journalier, suggestions cliquables
AJ-06	Pièce jointe au compte rendu	Étudiant	Dépôt .docx/.pdf jusqu'à 8 Mo, stockage en binaire dans le dépôt, texte extrait pré-rempli dans la zone de rédaction, téléchargement contrôlé
AJ-07	Anti-robot à la connexion	Système	reCAPTCHA v2 avec case à cocher sur l'écran de connexion, jeton vérifié côté serveur, refus motivé dans la sortie du serveur
AJ-08	Hors ligne automatique	Tous	Capture de toutes les semaines de l'année scolaire, filtrée par rôle, sans bouton à presser
AJ-09	Refonte des droits par matrice	Système	Une table unique `MATRICE[role][module]` pour 13 modules et 4 verbes, lue à la fois par les pages et par les API
AJ-10	Consultation de l'emploi du temps par salle	Tous	Filtre `salle_id` sur l'écran Emploi du temps, lien inverse depuis l'inventaire des salles

3. Besoins non fonctionnels

ID	Exigence	Implémentation
BNF-01	Application installable (PC et mobile)	Manifest `/manifest.webmanifest` : nom, icône 1024 px en versions « any » et « maskable », affichage standalone, thème `#4f46e5`. Enregistrement du service worker par un micro-composant client.
BNF-02	Utilisable hors connexion	Service worker maison, cache unique versionné `arp-v6` : navigations en « réseau d'abord, puis cache, puis page hors ligne », statiques en « cache d'abord », API jamais mise en cache sauf la capture.
BNF-03	Le hors ligne doit afficher le calendrier	La page `/offline` rend la grille 5 jours × 2 créneaux de la semaine courante ; un sélecteur relit la capture JSON pour les autres semaines de l'année.
BNF-04	Adapté au téléphone	Mise en page responsive : barre latérale `w-60` à partir de 768 px, barre de navigation basse défilante en dessous ; tableaux à défilement horizontal ; popup d'assistant plein écran sur mobile.
BNF-05	Sécurité de session	Cookie httpOnly, SameSite=Lax, signature HMAC-SHA256, durée 30 jours ; mots de passe hachés avec scrypt et sel aléatoire ; comparaison à temps constant.
BNF-06	Aucune exposition de requête après rechargement	Toute écriture répond par une redirection 303 ; le motif est détaillé en section 7.3.
BNF-07	Base ne demandant aucune administration	MongoDB lancé par le projet lui-même : `server-https.js` appelle le script de démarrage, surveille le port et relance au besoin ; schéma et index créés au premier appel, jeu de démonstration inséré si la base est vide.
BNF-08	Transparent sur le canal d'envoi réel	Courriels simulés et historisés dans une collection dédiée consultable ; la section 15 explique le passage à SMTP.
BNF-09	Langue	Interface intégralement en français, dates au format français, identifiants de code en français aussi (variables et commentaires).
BNF-10	Résilience vis-à-vis de l'IA	Chaque fonctionnalité « IA » a un repli local : assistant à règles, QCM construit sur les phrases du cours, recherche par recouvrement de mots. Sans clé, l'application reste complète.
BNF-11	Maîtrise du coût	Quota de 40 questions IA par compte et par jour, délai maximum de 45 secondes par appel, contexte limité à 4200 caractères, modèle réessayé sur erreur 400/404/429/5xx puis abandon.
BNF-12	Étanchéité des données entre comptes	Le snapshot envoyé au modèle est construit à partir de la session appelante ; une requête sur une autre classe est ignorée côté serveur ; les pièces jointes ne sont servies qu'à leur auteur, au professeur responsable et à l'admin.
BNF-13	Aucune requête SQL, donc pas d'injection	Les filtres sont construits en objets passés au driver (paramètres jamais concaténés dans une chaîne), y compris dans les recherches par mot-clé.

4. Matrice des autorisations

La source unique de vérité est `lib/permissions.ts` : une table `MATRICE[role][module]` associant à chaque couple un ensemble de verbes parmi `voir · creer · modifier · supprimer`. Les écrans appellent `await exiger(module, verbe)` — non connecté renvoie vers `/login`, droit absent renvoie vers le tableau de bord avec le motif « accès refusé » — et les routes d'API commencent par `if (!session || !peut(session.role, module, verbe)) return refus()`. Les deux niveaux lisant la même table, ils ne peuvent plus diverger.

> La matrice ne dit pas tout : la **propriété** est vérifiée en plus, action par action. Un professeur n'écrit que dans les matières dont il est responsable, un étudiant que dans son propre dépôt, son propre appel et sa propre réclamation.

Module	Admin	Professeur	Étudiant
Utilisateurs	voir · creer · modifier · supprimer	—	—
Classes	tout	voir	voir
Emails	tout	voir	voir
Salles	tout	voir	voir
Matières	tout	voir	voir
Emplois	voir · creer · modifier	voir	voir
Présences	voir	voir · creer · modifier	voir
Notes	voir	voir · creer · modifier	voir
Comptes rendus	voir	tout	tout
Réclamations	voir	tout	tout
Notifications	tout	voir	voir
Assistant	voir	voir	voir
Cours	— (module invisible)	tout	voir · creer (sa passation de quiz uniquement)

Decisions non évidentes, à assumer en soutenance :

- **Admin = consultation seule** sur notes, présences, comptes rendus et réclamations : il valide une matière mais n'écrit pas à la place du professeur, et seul l'auteur d'une réclamation pilote son statut.
- **Saisie réservée au professeur** : appel, notes CC/TP, clôture, correction de compte rendu, chapitres de cours.
- **Prof et étudiant ne voient pas Utilisateurs** — le lien est retiré de la barre latérale et la page refuse ; ils accèdent en revanche à Classes, Emails et à l'assistant.
- Le verbe `creer` côté étudiant sur Cours ne permet que de démarrer *sa* tentative de quiz, jamais de toucher un chapitre.

5. Modèle de données MongoDB

Dix-sept collections plus une collection technique `counters`. Le choix a été fait de conserver un modèle **relationnel** malgré Mongo : chaque document porte un `_id` **entier** distribué par un compteur atomique (`findOneAndUpdate` avec `$inc` et `upsert`), et les références entre documents restent des entiers. Cela rend la migration depuis SQLite directe, conserve des URLs lisibles (`/cours/12`) et permet aux écrans de rester inchangés. Les contraintes d'intégrité que Mongo ne vérifie pas nativement sont imposées par des **index uniques**.

La couche d'accès `lib/db.ts` expose six fonctions utilisées partout : `findDocs(collection, filtre, {sort, limit, skip})`, `findOne`, `countDocs`, `insertDoc`, `updateDoc` (retourne le nombre de documents touchés), `removeDocs`. À la lecture, `hydrate` renomme `_id` en `id` ; à l'écriture, `denormalize` fait l'inverse dans les filtres. Les dates sont stockées en texte local, au format `YYYY-MM-DD` ou `YYYY-MM-DD HH:mm` (`nowLocal()`), jamais en `Date` UTC : c'est ce qui rend les filtres par jour et par semaine prévisibles. `isDup` reconnaît le code Mongo 11000 et transforme le doublon en message d'erreur métier.

La connexion se relance elle-même : `getDb()` teste le port, et si `mongod` n'écoute pas, exécute le script de démarrage avant de retenter, ce qui explique qu'un redémarrage du PC suffise à faire repartir l'application complète. Les index sont créés à la première ouverture (`ensureIndexes`) et le jeu de démonstration est inséré dès que `users.estimatedDocumentCount()` vaut 0.

5.1 Vue d'ensemble des relations

Entité	Rôle	Relations
classes	Groupe d'étudiants d'une filière pour une année	1 — N users, 1 — N matieres, 1 — N emplois
users	Compte (admin, professeur ou étudiant)	N — 1 classes ; 1 — N matieres comme responsable ; 1 — N notes, presences, reclamations, cr_depots, quiz_passations
reset_tokens	Jeton d'invitation ou de changement de mot de passe	N — 1 users
emails	Boîte d'envoi (courriels simulés)	destinataire par adresse, sans clé étrangère
matieres	Enseignement avec coefficient, volume horaire et statut	N — 1 classes, N — 1 users, 1 — N emplois, notes, presences, cr_assignments, cours
salles	Lieu physique	1 — N emplois
emplois	Créneau d'une semaine donnée	N — 1 classes, N — 1 matieres, N — 1 salles
presences	Pointage d'un étudiant à une séance	N — 1 matieres, N — 1 users (étudiant), N — 1 users (professeur ayant relevé)
notes	Note CC, TP ou examen	N — 1 matieres, N — 1 users
cr_assignments	Compte rendu demandé avec date limite	N — 1 matieres, N — 1 users, 1 — N cr_depots
cr_depots	Remise d'un étudiant, avec pièce jointe binaire éventuelle	N — 1 cr_assignments, N — 1 users
reclamations	Réclamation ouverte par un professeur ou un étudiant	N — 1 users
notifications	Annonce globale créée par l'administration	sans destinataire : visible par tous
cours	Chapitre d'un support de cours, texte extrait + fichier d'origine	N — 1 matieres, N — 1 users (créateur), 1 — N cours_passages
cours_passages	Passage indexé pour la recherche sémantique	N — 1 cours, N — 1 matieres

quiz_passations	Tentative de QCM d'un étudiant, notée /20	N — 1 users, N — 1 matieres, N — 1 cours (nullable pour le QCM global)
assistant_usage	Compteur de questions IA par compte et par jour	N — 1 users

5.2 Dictionnaire détaillé

classes

Champ	Type	Contraintes	Description
id	Number	_id, distribué par counters	Identifiant
nom	String	unique, forcé en majuscules	Sigle du groupe, ex. INFO-2A
filiere	String	NOT NULL	Filière, ex. Informatique
annee	String	NOT NULL	Année scolaire, ex. 2026-2027

users

Champ	Type	Contraintes	Description
id	Number	_id	Identifiant
nom / prenom	String	NOT NULL	Identité affichée
email	String	unique	Identifiant de connexion, normalisé en minuscules
mot_de_passe	String ou null	NULL autorisé	`sel:hachage` scrypt ; null tant que l'invitation n'est pas activée
role	String	admin, prof ou etudiant	Rôle applicatif
classe_id	Number ou null	référence classes	Classe de l'étudiant ; null pour un professeur ou l'admin
actif	Number (0|1)	défaut 1	Compte désactivé = refus de connexion

reset_tokens · emails

Collection	Champs	Notes
reset_tokens	token, user_id, cree_le, utilise	Jetons de 48 caractères hexadécimaux aléatoires ; marqué consommé après usage, un jeton utilisé est refusé. Sans expiration temporelle.
emails	destinataire, sujet, contenu, lien, cree_le	Boîte d'envoi simulée ; `lien` commence toujours par `/`, ce qui permet le bouton « Ouvrir le lien » sans autoriser une URL externe.

matieres

Champ	Type	Contraintes	Description
code	String	unique, majuscules	Sigle, ex. DEVWEB
nom	String	NOT NULL	Libellé
coefficient	Number	défaut 1, interface 1–6	Pondération de la moyenne générale
heures	Number	défaut 6, interface ≥ 1	Volume horaire prévu
statut	String	en_cours, termine ou valide	Cycle de vie — voir section 6.2
prof_id	Number ou null	référence users	Responsable pédagogique
classe_id	Number	référence classes	Groupe concerné

salles · emplois

Collection	Champs	Contraintes

salles	nom, batiment, capacite, type	nom unique ; type parmi salle, amphi, labo
emplois	semaine_debut, jour, creneau, heure_debut, heure_fin, classe_id, matiere_id, salle_id	index unique `(classe_id, semaine_debut, jour, creneau)` — deux matières au plus par jour ; index simple sur `semaine_debut` pour les requêtes de semaine

Les horaires ne sont pas saisis : ils sont imposés par le créneau (1 = 09:00–12:00, 2 = 13:00–16:00).

presences · notes

Collection	Champs	Contraintes
presences	date, statut, matiere_id, etudiant_id, prof_id	statut parmi present, absent, retard ; index unique `(date, matiere_id, etudiant_id)` — l'enregistrement actualise au lieu de dupliquer ; index simple sur `etudiant_id`
notes	valeur, type, date, matiere_id, etudiant_id	valeur bornée 0–20 côté serveur, pas de 0,25 accepté ; type parmi CC, TP, EXAM ; index simple `(matiere_id, etudiant_id)` pour les moyennes

cr_assignments · cr_depots

Collection	Champs	Contraintes
cr_assignments	titre, consigne, matiere_id, prof_id, date_limite, cree_le	titre, consigne, matière et date limite obligatoires
cr_depots	assignment_id, etudiant_id, contenu, rendu_le, note, feedback, fichier, fichier_nom, fichier_type, taille_octets	index unique `(assignment_id, etudiant_id)` — un dépôt par étudiant et par sujet ; note null tant que non corrigée ; `fichier` est un binaire BSON (Buffer) et non une chaîne ; `fichier_type` vaut docx ou pdf

reclamations · notifications

Collection	Champs	Contraintes
reclamations	objet, message, type, statut, user_id, cree_le	type parmi note, emploi, salle, autre ; statut parmi ouvert, en_cours, traite, piloté par l'auteur
notifications	titre, message, lue, cree_le	`lue` est un champ réservé : les annonces étant globales, l'accusé individuel n'est pas exposé

cours · cours_passages · quiz_passations · assistant_usage

Collection	Champs	Contraintes
cours	matiere_id, numero, titre, texte, fichier_nom, fichier_type, fichier, taille_octets, terme, cree_par, cree_le, modifie_le	index unique `(matiere_id, numero)` ; numero de 1 à 30 ; texte d'au moins 300 caractères non blancs pour ouvrir le quiz ; `terme` (0|1) commande l'ouverture du quiz
cours_passages	cours_id, matiere_id, index, texte, vecteur	un document par passage de chapitre ; `vecteur` est un tableau de 768 nombres normalisés L2, vide si l'IA est indisponible ; index simples sur cours_id et matiere_id
quiz_passations	etudiant_id, matiere_id, cours_id, cle, titre, questions, reponses, note, justes, total, soumis, source, cree_le, fini_le	index unique `(etudiant_id, cle)` avec `cle` = `chapitre:<id>` ou `global:<matiere_id>` — c'est ce qui rend la seconde tentative impossible ; `questions` et `reponses` sont des tableaux embarqués
assistant_usage	user_id, jour, nb	un compteur par compte et par jour ; incrementé uniquement en cas de réponse IA réellement produite

5.3 Index et contraintes

Collection	Index	But

classes, users, matieres, salles	unique sur nom / email / code / nom	Empêcher le doublon métier, même si deux onglets soumettent en parallèle
emplois	unique (classe, semaine, jour, créneau) + simple semaine	Deux matières par jour maximum ; lecture de la grille
presences	unique (date, matière, étudiant) + simple étudiant	Un pointage par séance ; historique de l'étudiant
notes	simple (matière, étudiant)	Calcul des moyennes
cr_depots	unique (sujet, étudiant)	Un dépôt par étudiant
cours	unique (matière, numéro)	Numérotation sans trou
cours_passages	simples (cours, matière)	Récupération des passages d'un chapitre ou d'une matière
quiz_passations	unique (étudiant, clé) + simple matière	Une seule tentative ; tableau de bord du professeur

5.4 Jeu de démonstration initial

Au premier démarrage, `lib/db.ts` insère 2 classes, 1 administrateur, 3 professeurs, 5 étudiants, 4 matières, 3 salles, l'emploi du temps des semaines courante et précédente, des pointages et des notes CC/TP, 2 comptes rendus avec 2 dépôts dont un corrigé, 2 réclamations, 2 notifications et 1 courriel archivé. Tous les mots de passe du jeu valent `123456`. **Attention :** ce rappel est écrit sur l'écran de connexion, mais il n'est vrai que sur une base neuve ; après un envoi d'invitation ou un renvoi de mot de passe, seul le compte admin conserve un mot de passe connu, et il a pu être changé.

6. Règles de gestion et cycles de vie

6.1 Activation d'un compte

1. L'admin crée le compte (rôle professeur ou étudiant, classe pour un étudiant). Le mot de passe reste null.
2. La création génère un jeton, l'insère dans `reset_tokens` et consigne le courriel d'invitation contenant `/reset?token=…`.
3. Le destinataire ouvre le lien, saisit deux fois un mot de passe d'au moins 6 caractères.
4. Le mot de passe est haché, le jeton marqué consommé ; toute réutilisation du lien est refusée.
5. La connexion devient possible. Tant que le mot de passe n'est pas défini, elle est refusée avec un message évoquant le compte non activé.

6.2 Cycle de vie d'une matière

Statut	Déclencheur	Effet
en_cours	Création par l'admin	Saisie CC/TP autorisée au responsable ; appel possible ; création de comptes rendus et de chapitres possible ; dépôt étudiant possible ; les notes sont invisibles aux étudiants
termine	Clôture par le professeur, avec les notes d'examen de toute la classe	En une seule écriture : suppression des anciens EXAM de la matière, insertion des nouvelles notes, changement de statut. La matière entre dans la file « à valider » de l'admin. Aucun dépôt de compte rendu n'est plus accepté
valide	Bouton « Valider et publier aux étudiants » de l'admin	Uniquement depuis `termine`. Les notes deviennent lisibles par les étudiants, entrent dans la moyenne générale, et un courriel trace la publication. **Plus aucune écriture** sur les notes, les présences et les comptes rendus de la matière

Le retour en arrière n'est pas offert : le flux est volontairement à sens unique, pour que « validée » veuille dire archivée.

6.3 Cycle de vie d'un compte rendu

1. Le professeur crée le sujet avec date limite ; titre, consigne et matière sont obligatoires. L'admin consulte, il ne crée pas.
2. Avant la limite et tant que la matière est `en_cours`, chaque étudiant de la classe peut déposer une fois. Le dépôt accepte un texte, une pièce jointe .docx/.pdf, ou les deux.
3. Le dépôt reste modifiable et supprimable tant qu'il n'est pas noté et que le délai n'est pas dépassé.
4. Le professeur responsable consulte les dépôts, télécharge les pièces jointes, attribue une note 0–20 et un commentaire.
5. L'étudiant découvre note et retour dans son module Comptes rendus, avec le lien de sa propre pièce jointe.

6.4 Pièce jointe : ce qui est vérifié côté serveur

- Taille plafonnée à **8 Mo**, contrôle effectué sur la taille réelle du buffer avant tout traitement.
- Formats acceptés : **.docx et .pdf uniquement**, déterminés par la signature du fichier (PK pour le zip d'un docx, en-tête `%PDF` puis décodage effectif), pas par l'extension déclarée.
- Le nom du fichier est stocké tel quel (120 caractères maximum) mais nettoyé de tout guillemet, antislash et retour chariot avant d'être renvoyé en nom de téléchargement.
- Le fichier est rangé *dans* le document du dépôt, en binaire BSON : il n'y a aucun dossier de stockage à sauvegarder, et une suppression du dépôt supprime le fichier avec lui.
- Le texte extrait sert à pré-remplir la zone de rédaction, jamais à valider le dépôt : une pièce jointe scannée, dont l'extraction ne donne rien, reste acceptée et téléchargeable.

- Téléchargement soumis à la même matrice que la consultation, plus la propriété : l'étudiant ne reçoit que son dépôt, le professeur que les dépôts de ses sujets, l'admin tous.

6.5 Droits de manipulation vérifiés côté serveur

- Pages réservées : `/users` exige le module Utilisateurs, que la matrice ne donne qu'à l'admin ; un professeur ou un étudiant qui l'appelle directement est renvoyé vers le tableau de bord avec le motif.
- Emploi du temps : écriture admin, et chaque créneau soumis est re-vérifié (matière appartenant bien à la classe, salle existante) — une soumission fabriquée à la main est refusée.
- Notes : un professeur ne saisit que dans ses matières, pour un étudiant inscrit dans la classe de la matière, et tant que la matière est « en cours ». La clôture appartient au professeur ; l'administrateur ne fait que valider.
- Présences : appel limité aux matières dont le professeur a la charge ; il ne supprime que les lignes de ses matières, et uniquement tant que la matière n'est pas validée.
- Comptes rendus : création réservée au professeur ; dépôt étudiant limité à sa classe, avant la date limite, une seule fois ; correction réservée au professeur responsable.
- Cours : chapitre créé et modifié par le professeur responsable de la matière ; un élève ne peut ni le modifier ni fermer le chapitre ; le quiz ne peut être démarré que si le chapitre est marqué « terminé ».
- Notifications : création et suppression admin uniquement. Réclamations : création par professeur ou étudiant, statut piloté par l'auteur, consultation admin.

6.6 Règles de l'emploi du temps

- Deux créneaux par jour, cinq jours : dix lignes au maximum par classe et par semaine.
- Enregistrer une semaine remplace les créneaux de cette semaine pour la classe ; un créneau laissé vide devient libre.
- Une salle est obligatoire dès qu'une matière est placée.
- La semaine précédente peut être copiée puis ajustée.
- **Semaine verrouillée** : à partir du samedi 00:00, plus aucune écriture sur la semaine ; la fenêtre d'édition disparaît et un encart propose de préparer la semaine suivante.

6.7 Calcul des moyennes

Moyenne de matière = moyenne arithmétique des notes publiées (CC, TP et examen). Moyenne générale = moyenne pondérée des moyennes de matière par leur coefficient, sur les seules matières `valide`. Affichage coloré selon le seuil 10/20. Le QCM n'entre dans aucune de ces moyennes : sa note est stockée dans `quiz_passations`, consultée séparément.

7. Architecture technique

7.1 Organisation du code

Élément	Rôle
`app/**/page.tsx` (18)	Composants serveur : lecture directe de MongoDB et rendu HTML. Aucun appel réseau client, sauf l'assistant et le sélecteur de semaine hors ligne
`app/api/**/route.ts` (22)	Points d'entrée d'écriture et de lecture binaire : analyse du formulaire, contrôle du rôle et de la propriété, écriture, redirection
`lib/db.ts`	Ouverture unique et mémoïsée, distribution des identifiants, création des index, jeu de démonstration, hachage et vérification des mots de passe
`lib/permissions.ts`	Matrice des droits, `peut()` pour les API, `exiger()` pour les pages
`lib/auth.ts`	Session (cookie signé), connexion, jeton d'invitation, définition du mot de passe par jeton
`lib/filtres.ts`	Lecture des paramètres d'URL, filtres de période et d'année, pagination, construction des liens `?retour=`
`lib/retour.ts`	`routeurRetour()` : la seule façon de répondre après un POST, avec ses quatre helpers `vers / erreur / ok / refus`
`lib/semaine.ts`	Dates locales, lundi et dimanche de semaine, libellé, règle de verrouillage du samedi
`lib/origin.ts`	Restitution de l'origine réelle de la requête pour que les redirections conviennent aussi au téléphone
`lib/cours.ts`	Extraction de texte .docx et .pdf, découpage en passages, seuil de texte suffisant
`lib/rag.ts`	Indexation sémantique, similarité cosinus, repli par recouvrement de mots, mélange déterministe
`lib/qcm.ts`	Fabrication des questions (IA puis règles locales), correction et note /20
`lib/gemini.ts`	Client REST commun : génération de contenu et plongements, liste de modèles, délais et replis
`lib/assistant*.ts` (3)	Moteur local à règles, refus connus, quota, orchestration IA, construction du contexte par compte
`lib/hors-ligne.ts`	Emplois du temps de toute l'année scolaire, réduits au périmètre du compte appelant
`lib/recaptcha.ts`	Vérification du jeton v2 auprès de `siteverify`, activation par les clés
`lib/pdf.ts`	Générateur PDF maison : PDF 1.4, polices Helvetica de base, encodage WinAnsi pour les accents, calcul des largeurs de glyphes, page paysage ; aucune dépendance externe
`lib/mail.ts`	Consignation d'un courriel dans la collection `emails` (simulation de l'envoi)
`middleware.ts`	Garde-frontière unique : bloque pages et API sans cookie valide, laisse passer cinq chemins publics
`components/` (13)	Sidebar, Modal, ConfirmForm, Filtres, Pagination, Stats, EmploiTableau, HorsLigneSemaines, AssistantChat, AssistantFlottant, DepotJoint, PrintButton, PwaRegister
`public/`	Manifest, service worker, icône
`server-https.js`	Démarrage HTTPS de la version produite, port 443, certificat auto-adaptatif, lancement de mongod

7.2 Flux d'une écriture

`<form method="post" action="/api/x">` → le middleware vérifie la forme du cookie → la route handler lit `req.formData()`, contrôle le rôle puis la propriété pédagogique, écrit via la couche `lib/db.ts` (objets de filtre passés au driver, jamais de concaténation), appelle `revalidatePath()` sur les pages concernées, puis répond 303 par `routeurRetour()`. Le formulaire POST emporte au passage `?retour=` contenant les filtres d'URL du moment : l'utilisateur revient donc exactement sur l'onglet, la classe, la période et la page de résultats d'où il est parti.

7.3 Pourquoi 303 plutôt qu'un autre code

Next 13 applique `NextResponse.redirect()` en 307 par défaut, ce qui invite le navigateur à rejouer le POST sur le chemin de la page. La page étant un composant serveur, le corps du formulaire n'est plus recevable : on obtenait une erreur 500 après chaque connexion et chaque formulaire. Tous les points d'écriture répondent donc explicitement en 303, qui force un rechargement en GET. Cette précision a été la source du plus grand nombre de défauts rencontrés.

7.4 Rendu, révalidation et composants client

Les écrans portant `export const dynamic = "force-dynamic"` sont rendus à la demande : les données affichées ne peuvent pas être périmées. Après une écriture, `revalidatePath` purge les pages touchées par la même requête. Neuf composants sont marqués `"use client"`, et toujours pour la même raison : un comportement qui a besoin du navigateur (dialog natif, `window.confirm`, `window.print()`, enregistrement du service worker, zone de défilement, pré-remplissage après extraction). Aucun de ces composants n'émet de requête d'écriture : les mutations restent des POST HTML ordinaires. Le widget reCAPTCHA, lui, n'a besoin d'aucun composant : c'est le script de Google qui écrit le champ du formulaire.

7.5 Particularités du serveur Node

L'application n'est pas servie par `next start` mais par `server-https.js`, un serveur `https` Node devant `next({dev}).getRequestHandler()`. Trois conséquences sont écrites dans le code :

- **Le mode production est figé au build.** Une route ajoutée depuis le dernier `npm run build` renvoie 404 ou reste suspendue : `npm run build` puis relance sont obligatoires.
- **`req.nextUrl.origin` n'est pas l'origine du client.** Il vaut l'adresse interne de Next ; sans `clientOrigin(req)`, toute redirection depuis un téléphone renverrait sur un `localhost` injoignable.
- **Le port est fixé à 443**, sans repli : une seule URL, sans numéro de port à dicter. Le serveur écoute sur `0.0.0.0`, l'adresse affichée au démarrage est relue sur les interfaces réseau en écartant les cartes virtuelles.

8. Interface HTTP

8.1 Pages

Chemin	Contenu	Accès
/	Tableau de bord selon le rôle : 4 à 5 indicateurs, calendrier de la semaine 5×2, trois dernières notifications, accès au hors ligne	connecté
/login	Connexion, bandeaux d'erreur, d'activation et d'anti-robot, rappel des comptes de démonstration, case à cocher reCAPTCHA	public
/reset	Définition du mot de passe depuis un jeton	public
/offline	Bandeau « Vous êtes hors ligne », grille de la semaine courante, sélecteur des autres semaines relu depuis le cache	public
/users	Liste avec onglets par rôle, recherche, invitation, renvoi, activation/désactivation, suppression	admin
/classes	Grille ou liste par année, fiche de classe avec effectif et emploi du temps, affectation par cases à cocher	tous en lecture, écriture admin
/emails	Boîte d'envoi avec lien activable ; les non-admins ne voient que les messages qui leur sont adressés	tous en lecture, écriture admin
/salles	Inventaire, capacité, type, créneaux utilisés et matières qui y vont	tous en lecture, écriture admin
/matieres	Liste filtrée par année, classe et statut, bandeau de stats, file de validation, création avec notification du professeur	tous en lecture, écriture admin
/emplois	Grille hebdomadaire, choix classe et semaine, éditeur en popup, copie de la semaine précédente, PDF, impression, encart de semaine verrouillée	tous en lecture, écriture admin
/presences	Appel par matière et date pour le professeur, matrice étudiants × matières avec taux de présence, historique détaillé	tous en lecture, écriture prof
/notes	Saisie CC/TP en popup, grille par étudiant, clôture avec notes d'examen, moyenne publiée pour l'étudiant, moyenne générale pondérée	tous en lecture, écriture prof
/comptes-rendus	Sujets avec date limite, dépôts, zone de rédaction avec pièce jointe et extraction, correction, restitution à l'étudiant	prof et étudiant
/cours	Chapitres par matière, import de support, ouverture et fermeture du quiz, copie de quiz par chapitre, QCM global	prof et étudiant
/cours/[id]	Chapitre : texte, support téléchargeable, mode de recherche utilisé, édition, résultat du quiz et navigation vers le chapitre précédent/suivant	prof et étudiant
/cours/quiz	Trois états : règles et bouton démarrer, questionnaire à cocher, copie corrigée avec détail par question	étudiant
/reclamations	Création et suivi de statut par l'auteur, consultation admin, stats par statut	tous
/notifications	Publication par l'admin, consultation globale	tous en lecture, écriture admin

8.2 Opérations d'écriture

Point d'entrée	Méthode	Actions	Habilitation effective
/api/auth/login	POST	ouverture de session	public, après passage de l'anti-robot si configuré
/api/auth/logout	POST	fermeture, et purge de la capture hors ligne côté navigateur	connecté

/api/auth/reset	POST	définition du mot de passe par jeton	public
/api/users	POST	create, update, renvoyer, toggle, delete	admin ; jamais sur le compte admin ni sur soi
/api/classes	POST	create, update, affecter, delete	admin
/api/emails	POST	send, delete	admin ; lien interne obligatoire
/api/salles	POST	create, update, delete	admin ; suppression refusée si la salle est utilisée
/api/matieres	POST	create, update, valider, delete	admin
/api/emplois	POST	save-week, copy-prev	admin, semaine non verrouillée
/api/emplois/pdf	GET	PDF de la semaine, en pièce jointe	connecté, étudiant forcé à sa classe
/api/presences	POST	save (appel), delete	prof pour ses matières, matière non validée
/api/notes	POST	create (CC/TP), update, cloturer (EXAM), delete	prof responsable ; gel après validation
/api/comptes-rendus	POST	create, delete, noter · depot, update_depot, delete_depot	prof pour create/noter/delete, étudiant pour depot ; multipart accepté
/api/comptes-rendus/extraction	POST	extraction de texte d'une pièce jointe, sans écriture	droit « creer » sur comptes rendus
/api/comptes-rendus/fichier	GET	téléchargement de la pièce jointe d'un dépôt	auteur, professeur responsable ou admin
/api/reclamations	POST	create, update (statut), delete	auteur uniquement
/api/notifications	POST	create, update, delete	admin
/api/cours	POST	create, update, terminer, rouvrir, delete	prof responsable de la matière
/api/cours/fichier	GET	téléchargement du support d'un chapitre	professeur du chapitre ou étudiant de la classe
/api/cours/quiz	POST / GET	demarrer, soumettre ; GET = métadonnées de la copie	étudiant ; le corrigé n'est jamais envoyé avant la soumission
/api/assistant	POST	question à l'assistant	les trois rôles, lecture seule, 403 si le module est refusé
/api/offline-data	GET	capture de l'emploi du temps de l'année	connecté ; 401 sans session, ce qui efface la capture

9. Sécurité et anti-robot

Domaine	Mise en œuvre
Mots de passe	scrypt 32 octets avec sel de 16 octets par compte, stocké sous la forme `sel:hachage` ; vérification par comparaison à temps constant ; jamais de mot de passe en clair stocké ni affiché. Minimum 6 caractères à l'activation.
Session	Cookie `arp_session` = `id.signature`, signature HMAC-SHA256 tronquée à 16 hex, httpOnly, SameSite=Lax, chemin racine, 30 jours. La clé vient de `ARP_SECRET`. Le middleware vérifie la forme du cookie pour toutes les pages et API non publiques ; chaque route recontrôle la signature en relisant la session.
Invitations	Jeton de 24 octets aléatoires en hexadécimal, stocké, marqué consommé après usage. *Limite assumée : pas d'expiration temporelle.*
Contrôle d'accès	Deux niveaux lisant la même table : `exiger()` côté écran, `peut()` + vérification de propriété côté écriture. Le masquage d'interface n'est jamais la garantie ; un 403 ou une redirection motivée répond toujours.
Injections	Aucune chaîne SQL : les filtres sont des objets passés au driver MongoDB. Les champs injectables (nom de fichier, contenu, consigne) sont traités comme des valeurs, jamais comme du code ou du nom de collection.
XSS	React échappe les interpolations ; les retours à la ligne sont rendus par la classe CSS `whitespace-pre-wrap` et non par injection HTML ; les liens de téléchargement passent par une route, jamais par une URL fabriquée depuis le nom du fichier.
Fichiers	Extension et signature vérifiées, taille plafonnée avant traitement, nom nettoyé pour l'en-tête `Content-Disposition`, binaire stocké en base et renvoyé au compte par une route qui applique la matrice des droits.
Secrets hors dépôt	`.gitignore` exclut `node_modules/`, `.next/`, `data/`, `certs/` (cle privée TLS), `.assistant-secret`, `.tunnel-secret` et les fichiers `.env*` depuis le 7 novembre 2026. Aucune clé n'est écrite dans le code.
Confidentialité par compte	Le contexte envoyé au modèle IA est construit à partir de la session et ne contient que ce que ce compte a le droit de voir ; notes et pièces jointes d'un autre étudiant ne peuvent pas être obtenues par une requête fabriquée.

9.1 Anti-robot reCAPTCHA v2

Le module est un **widget à case à cocher** (« Je ne suis pas un robot »), type v2. La version du code suit le type de clé créé dans la console Google : les deux modes ne sont pas interchangeables, et servir le mauvais markup provoque une erreur visible. Une v3 par score a été essayée puis écartée : elle demandait un composant client pour appeler `grecaptcha.execute()`, ne produisait aucun jeton dans un onglet masqué, et sa décision était invisible. Le widget rendu accepte un contrôle montré tel quel en soutenance.

- **Activation automatique.** `recaptchaActif()` ne renvoie vrai que si `RECAPTCHA_SITE_KEY` et `RECAPTCHA_SECRET` sont tous les deux non vides. Sans les deux, la page de connexion redevient strictement classique : le code n'a pas besoin d'être modifié pour une démonstration sans réseau.
- **Collecte du jeton.** Le script `api.js`, chargé en `afterInteractive` et **sans** paramètre `render=`, remplace la div `g-recaptcha` placée dans le formulaire par le iframe du widget, et y crée lui-même le champ `g-recaptcha-response` au moment du clic. Aucun composant client n'est nécessaire : le formulaire reste un POST HTML ordinaire, rien n'est intercepté.
- **Vérification serveur.** La route de connexion appelle `siteverify` avec le secret, le jeton, l'adresse IP cliente (premier élément de `x-forwarded-for`, seule source exploitable derrière un proxy) et le `origin` public. Cet `origin` attache le jeton à la clé du site : un jeton volé sur un autre domaine est refusé. Seule la réponse `success` est exigée — la v2 ne renvoie ni score ni action.
- **Refus fermé par défaut.** Jeton absent, jeton expiré (ils le sont au bout de deux minutes), `origin` discordant ou `siteverify` injoignable : la connexion est refusée, et le test du mot de passe n'est même pas exécuté. Le motif est écrit dans la sortie du serveur et jamais affiché au visiteur, pour ne pas donner à

un robot l'information qui lui permettrait d'ajuster sa méthode ; l'écran montre seulement « vérification anti-robot échouée, recochez la case ».

- **Domaines.** Google n'émet de jeton que pour les hôtes déclarés dans la console. Chaque adresse réellement utilisée — localhost, l'IP du poste, l'hôte du tunnel — doit y figurer, sinon le widget affiche une erreur et la vérification échoue sur ce domaine.

9.2 Limites de sécurité à connaître

- La clé HMAC a une valeur par défaut de démonstration si `ARP_SECRET` est absent : en exposition réelle, une session pourrait être fabriquée par un tiers connaissant le schéma.
- Les comptes de démonstration partagent un mot de passe faible, volontairement rappelé sur l'écran de connexion.
- `certs/` contient une clé privée ; le dépôt public ne doit jamais la contenir.
- Le dépôt en ligne est public : tant que le repli `ARP_SECRET` existe et que le mot de passe admin de départ n'a pas été changé, l'exposition de l'adresse réelle reste un risque.
- Aucune expiration temporelle des jetons d'invitation, aucune limitation de tentative à la connexion en dehors de reCAPTCHA, pas de journal d'audit des accès.

10. Module Cours : documents, RAG et QCM noté /20

Ce module n'était pas demandé par le cahier des charges. Il répond à une question simple : un étudiant qui a son cours sur son téléphone doit pouvoir être interrogé dessus, et corrigé tout de suite, sans que le professeur ait à fabriquer un questionnaire à la main.

10.1 Du fichier au chapitre

Le professeur choisit une matière, un numéro de chapitre (1 à 30) et dépose un **.docx** ou un **.pdf** (8 Mo maximum). Le texte est extrait côté serveur par `lib/cours.ts`, sans aucune dépendance ajoutée :

- **.docx** : lecture manuelle de la table centrale du zip (recherche de la signature de fin jusqu'à 66 000 octets en arrière), puis du membre `word/document.xml`, dont les puces et les fins de paragraphe sont converties en sauts de ligne.
- **.pdf** : décompression des flux (Deflate, raw et inflate), lecture des tables **ToUnicode** de type `bfchar` et `bfrange` pour retrouver les vrais caractères, puis parcours des opérateurs de texte `Tj` et `TJ` dans les blocs `BT…ET` ; l'ordre des arguments est respecté pour ne pas inverser les fragments kernés.
- Le fichier d'origine est conservé en binaire dans le document du chapitre et re-téléchargeable, avec les mêmes règles d'accès que les pièces jointes de comptes rendus.

Un chapitre dont le texte utile dépasse **300** caractères non blancs peut être marqué « terminé », ce qui ouvre son quiz. En dessous, le quiz reste fermé : un cours réduit à un titre ne peut pas produire de questions pertinentes.

10.2 Indexation sémantique

À l'enregistrement, le texte est découpé en passages de **800 caractères avec 120 de recouvrement**, en ignorant les blocs de moins de 40 caractères ; le recouvrement évite qu'une réponse coupée en deux ne devienne introuvable. Chaque passage est envoyé aux plongements Gemini (`gemini-embedding-001`, 768 dimensions demandées sur les 3072 natives, par lots de 16, texte réduit à 8 000 caractères) puis stocké dans `cours_passages` après normalisation L2.

La recherche est faite par **similarité cosinus calculée en JavaScript** : le serveur MongoDB local ne fournit pas `$vectorSearch`, et les chapitres de démonstration produisent quelques dizaines de passages, ce qui rend le calcul complet immédiat. Les **4 meilleurs passages** sont retenus. Deux replis existent :

- si un seul passage n'a pas de vecteur (clé IA absente, appel raté), la recherche retombe sur un recouvrement de mots ;
- l'écran du chapitre affiche explicitement le mode utilisé — « recherche sémantique (index Gemini) » ou mots-clés — pour qu'on ne prenne pas le repli pour du sémantique.

10.3 QCM à la volée, une seule tentative

Quand le chapitre est terminé, l'étudiant de la classe voit « Passer le quiz » : **6 questions** pour un chapitre, **8 questions** pour le QCM global d'une matière, ce dernier n'étant proposé que lorsque tous les chapitres sont terminés.

Point	Mécanisme réel
Fabrication	Gemini rédige les questions à partir des seuls passages fournis ; le résultat n'est accepté que s'il contient au moins `min(3, nb)` questions valides. Sinon, un moteur local prend le relais : la bonne réponse est une phrase réellement extraite du chapitre (40 à 220 caractères), les distracteurs sont des phrases d'autres passages du même cours
Personnalisation	Un tirage aléatoire est mélangé avec une graine dérivée de l'identifiant de l'étudiant : deux élèves du même chapitre n'obtiennent pas le même ordre de questions

Une seule tentative	Index unique `(etudiant_id, cle)` avec `cle` = `chapitre:<id>` ou `global:<matiere_id>` ; le document est créé non soumis, la soumission est un `updateOne` filtré sur `soumis: 0`, donc une double soumission ne double pas la note
Corrigé	Il n'est jamais envoyé au navigateur avant la soumission : la route GET ne renvoie que les métadonnées de la copie
Note	`arrondi(justes / total × 80) / 4`, donc une note sur 20 au pas de 0,25, calculée côté serveur. Elle est stockée dans `quiz_passations` et **ne rejoint pas la collection `notes`** : elle ne modifie ni la moyenne de matière ni la moyenne générale
Restitution	Score, nombre de bonnes réponses, badge « chapitre maîtrisé » à partir de 12/20, puis revue question par question avec la réponse donnée, la bonne et la mention « Sans réponse. »
Tableaux de bord	Le professeur voit copies, moyenne et taux de réussite par chapitre et pour le QCM global ; l'étudiant ne voit que sa copie
Protection	Un chapitre n'est plus supprimable ni réouvrable si des élèves ont déjà composé avec

11. Assistant : Gemini avec repli local

Bulle flottante en bas à droite, présente pour les trois rôles dès que la matrice autorise le module, absente sur la page hors ligne. L'assistant est **en lecture seule par construction** : sa route d'API ne fait qu'appeler des fonctions de consultation, il n'existe aucun chemin d'écriture dans ce module.

11.1 Trois niveaux de réponse

1. **Refus connus d'abord**, avant même de consulter le modèle : cinq familles détectées sur des mots-clés normalisés — altérer l'application, demander des identifiants ou mots de passe, modifier ses propres notes ou absences (étudiant), se modifier soi-même ou sa classe affectée (professeur), et toucher les données d'un autre étudiant ou d'une autre classe.
2. **Gemini ensuite**, si une clé existe et si le quota du jour n'est pas épuisé.
3. **Moteur local enfin** : règles par thème (prochaine séance, notes, présences, comptes rendus, matières, réclamations, notifications, salles, aide) qui lisent la base directement. Ce niveau suffit à lui seul pour la démonstration.

11.2 Étanchéité et coût

La vraie barrière n'est pas la consigne : **le modèle ne voit que ce qui lui est envoyé**. Le contexte est un snapshot de 4 200 caractères maximum, construit à partir de la session appelante et jamais d'un identifiant passé en paramètre : identité du compte, emploi du temps de la semaine (60 lignes), matières accessibles (30), statistiques du rôle, comptes rendus (20), réclamations (12, l'admin voit tout, les autres leurs propres messages), cinq dernières notifications. Les mots de passe et les adresses électroniques n'y entrent pas. L'historique est tronqué aux 6 derniers messages et la question à 300 caractères.

Réglage	Valeur	Pourquoi
Quota	40 questions IA par compte et par jour (`ASSISTANT_QUOTA`), comptées dans `assistant_usage`	Le compteur n'est incrémenté qu'en cas de réponse IA réellement produite ; le repli local est gratuit
Modèles essayés par ordre	`gemini-3.5-flash` , puis `gemini-3.1-flash-lite` , puis `gemini-2.5-flash` ; un modèle absent de la liste peut être imposé en ligne 2 du fichier de clé	Erreur 400, 404, 429 ou 5xx et timeout font passer au modèle suivant ; deux identifiants longtemps utilisés renvoient désormais « no longer available to new users » et faisaient retomber silencieusement en local
Délai	45 secondes, en budget global et par appel	Un élève ne doit jamais attendre plus ; au-delà, la réponse locale part
Sortie	température 0,5, 1 600 tokens de sortie, `thinking_budget: 0`	Sans budget de réflexion nul, les tokens de « raisonnement » remplissaient la sortie et coupaient la réponse avant sa fin
Recharge de la clé	lue une seule fois au chargement du module	Changer la clé ou le modèle exige un redémarrage du serveur

11.3 Deux régimes de réponse et les suggestions

Le prompt distingue deux situations, ce qui évite de transformer l'assistant en base de questions fermées :

- **Régime A — scolarité du compte** : ancrage strict sur le snapshot, rien en dehors.
- **Régime B — travail académique libre** : plan de compte rendu, fiche de révision, explication d'une notion, reformulation, QCM. Le modèle peut mobiliser ses propres connaissances, en s'appuyant sur les matières et coefficients réels, et doit livrer une *trame* plutôt qu'un devoir rédigé clé en main.

La consigne de fin demande une ligne `PROPOSITIONS: q1 | q2 | q3`. Elle est retirée du texte affiché et décodée en trois puces cliquables sous la bulle concernée, ce qui relance la conversation sans zone de saisie. Les questions préparées par rôle restent disponibles sous la zone de saisie. Le pied de chaque réponse indique la source réelle — modèle Gemini avec le quota restant, ou règles internes — pour qu'un élève ne confonde pas les deux, et rappelle que les réponses sont produites à partir de ses seules données accessibles.

12. PWA : installation et mode hors ligne

12.1 Comment l'installation est rendue possible

Les trois conditions du navigateur sont remplies : manifest valide avec icône et affichage standalone, service worker actif avec gestionnaire de requêtes, contexte sécurisé. Sur PC, l'origine `localhost` est réputée sécurisée, ce qui permet d'installer sans certificat. Sur Android, `localhost` n'existe pas pour le téléphone et une adresse locale en HTTP n'est pas un contexte sécurisé : le HTTP y rend l'installation et le hors ligne impossibles. C'est la raison du serveur HTTPS local.

> Point vérifié et contre-intuitif : **Chrome/Android n'accepte pas une autorité de certification installée par l'utilisateur** pour un site servi en HTTPS local. Sur certains téléphones, l'installation exige donc un tunnel public (Cloudflare Quick Tunnel, URL `trycloudflare.com`) plutôt que l'IP du poste. Le certificat local reste néanmoins nécessaire pour l'usage depuis d'autres PC du réseau et pour éviter tout avertissement sur le poste lui-même.

12.2 Stratégie de cache du service worker

Un seul cache, versionné `arp-v6` : une nouvelle publication évince automatiquement les entrées anciennes, ce qui règle le problème du déploiement d'un service worker modifié.

Type de requête	Stratégie	Comportement constaté
Fichiers essentiels (manifest, icône, `/login`)	Pré-cachés à l'installation	Toujours disponibles
Navigation HTML	Réseau d'abord, puis cache de la page `/offline`, puis `/login`	Page déjà vue servie hors ligne ; sinon message hors ligne avec le calendrier
Ressources statiques	Cache d'abord, puis réseau	Chargement instantané, pas de consommation de données
`/api/offline-data`	Réseau d'abord, puis capture cachée ; 504 JSON si aucune capture	Le sélecteur de semaine fonctionne hors ligne
Tout autre `/api/…`	Réseau uniquement, jamais mis en cache	Une écriture n'est jamais simulée hors ligne

12.3 Capture automatique, sans bouton

Le principe : ne rien demander à l'utilisateur. `lib/hors-ligne.ts` construit **toutes les semaines de l'année scolaire** (du lundi qui suit le 1er août au 31 juillet suivant), groupées semaine par semaine puis classe par classe, réduites au périmètre du rôle — toutes les classes pour l'admin, ses séances pour le professeur, sa classe pour l'étudiant — et la semaine courante est toujours présente.

- `/api/offline-data` est en `force-dynamic` et `no-store` ; il répond 401 sans session.
- La capture est déclenchée à l'installation, à l'activation, et en `event.waitUntil()` sur **toute navigation en 2xx**, avec un throttle de 30 secondes : ouvrir un écran connecté suffit à sauvegarder.
- Un 401 sur `/api/offline-data` **efface la capture** : se déconnecter retire les données de l'appareil.
- Le cache contient deux entrées : la page `/offline` et le JSON `/api/offline-data`.

12.4 Ce qui marche hors ligne, ce qui ne marche pas

Marche : toute l'emploi du temps de l'année pour le périmètre du compte, affiché en grille 5 jours × 2 créneaux, avec changement de semaine depuis le cache, et l'impression navigateur. **Ne marche pas :** les écritures (appel, dépôt, note, création) et les pages de consultation autres que l'emploi du temps. Le service worker ne met volontairement rien en file d'attente à rejouer : un pointage non confirmé silencieusement serait trompeur pour l'enseignant.

12.5 Accès depuis un téléphone

1. `npm run build` puis `npm start` : le serveur écoute sur le port 443 et affiche l'adresse à utiliser.
2. Installer `certs/arp-root-ca.pem` comme autorité de certification sur le téléphone — ou, si Chrome refuse, ouvrir l'URL d'un tunnel public.
3. Ouvrir l'adresse affichée (aucun numéro de port), puis menu Chrome → « Installer l'application ».
4. Consulter une fois l'emploi du temps : la capture hors ligne se fait seule.

Le certificat est produit par mkcert (autorité locale créée sur le poste). Il n'y a plus rien à régénérer à la main : `scripts/ensure-cert.js` relit les adresses IPv4 privées du poste, resigne si elles changent, et applique le nouveau contexte au serveur en cours d'écoute (contrôle toutes les 30 secondes). L'autorité racine ne change pas, donc le téléphone n'a pas besoin de la réinstaller après un changement d'IP ; en revanche l'origine ayant changé, le raccourci PWA doit être recréé.

13. Installation, démarrage, démonstration

13.1 Prérequis et commandes

Besoin	Commande	Résultat
Installation	`npm install`	dépendances du projet
Développement (rechargement à chaud)	`npm run dev`	`http://localhost:3000` — non utilisable depuis le téléphone
Base de données	`npm run db:start`, ou rien du tout	le serveur HTTPS démarre mongod lui-même si le port 27017 ne répond pas
Production locale et téléphone	`npm run build` puis `npm start`	`https://localhost` et `https://<IPv4 du poste>`, sans numéro de port
Inspection des collections	`npm run db:tables`	vidage des collections en tableaux, mot de passe masqué
Migration depuis l'ancienne base SQLite	`npm run db:migrate`	refusé si la base n'est pas vide, sauf `--force` ; conserve les identifiants numériques
Import des chapitres de démonstration	`npm run cours:gen` puis `npm run db:cours`	25 chapitres .docx puis import réel par l'API, avec extraction, droits et indexation
Réinitialiser les données	purger les collections, ou repartir d'une base vide	le jeu de démonstration est recréé au premier appel. **Ne pas utiliser `npm run db:migrate --force` pour un simple nettoyage : cela remettrait le mot de passe admin à 123456**

13.2 Variables d'environnement

Variable	Défaut si absente	Effet
`MONGO_URL` / `MONGO_DB`	`mongodb://127.0.0.1:27017` / `arp_education`	Base utilisée ; une URL distante désactive le démarrage automatique de mongod
`ARP_SECRET`	repli de démonstration, ou le fichier `.tunnel-secret`	Clé de signature des cookies de session ; à fixer absolument hors développement
`RECAPTCHA_SITE_KEY` / `RECAPTCHA_SECRET`	vides	Les deux sont nécessaires pour activer l'anti-robot ; sinon la connexion reste classique
`GEMINI_API_KEY` (ou `.assistant-secret`)	absente	Active l'assistant IA, les plongements RAG et la fabrication de QCM par modèle ; sinon repli local intégral
`GEMINI_EMBEDDING` / `GEMINI_DIMENSIONS`	`gemini-embedding-001` / `768`	Modèle et dimension d'indexation ; changer de dimension oblige à ré-indexer
`ASSISTANT_QUOTA`	`40`	Questions IA par compte et par jour
`HTTPS_PORT` / `HTTPS_DEV` / `WIFI_IP`	`443` / off / détection	Port d'écoute, mode développement du serveur, adresse forcée pour le certificat
`MONGOD_EXE` / `MONGOD_CFG`	recherche dans `Tools\mongodb`	Emplacement du binaire et configuration utilisés par le lanceur automatique

13.3 Comptes de démonstration

Rôle	Adresse électronique	Remarque
Administrateur	`admin@arp.edu`	Compte fixe, seul non créé par l'application

Professeur	`sami.benali@arp.edu`	Responsable de DEVWEB et BDD (INFO-2A)
Professeur	`nadia.trabelsi@arp.edu`	Responsable de COMPTA (GEST-2A)
Étudiant	`ahmed.j@arp.edu`	Classe INFO-2A

> **Avertissement pour la démonstration.** L'encart de la page de connexion annonce le mot de passe `123456` pour ces quatre comptes. Ce n'est vrai que sur une base tout juste seedée. En l'état, sur la base de travail, le compte admin ne répond plus à ce mot de passe : vérifier avant de montrer l'écran, ou réactiver un compte de démonstration explicite. Afficher un mot de passe qui ne marche pas pendant une soutenance est le genre de détail qui coûte cher.

13.4 Scénario de démonstration conseillé (12 minutes)

1. Admin : créer un étudiant par son courriel, montrer que son compte apparaît « à définir ».
2. Admin : ouvrir Emails, cliquer le lien d'activation, définir le mot de passe.
3. Étudiant : se connecter, consulter l'emploi du temps de la semaine, le télécharger en PDF.
4. Prof : faire l'appel sur une séance, ajouter deux notes CC/TP, créer un compte rendu avec date limite.
5. Étudiant : déposer son compte rendu **avec une pièce jointe**, montrer le texte repris automatiquement dans la zone de rédaction et le lien 📎.
6. Prof : télécharger la pièce jointe de l'étudiant, la noter.
7. Prof : importer un chapitre .docx dans Cours, montrer l'extraction et l'indexation ; marquer le chapitre terminé.
8. Étudiant : passer le QCM du chapitre, obtenir sa note /20 immédiate et la revue question par question ; montrer qu'une seconde tentative est impossible.
9. Étudiant : poser une question libre à l'assistant, montrer les suggestions cliquables et la source affichée ; poser une question interdite et montrer le refus.
10. Prof : clôturer la matière ; Admin : la valider, montrer que la moyenne publiée apparaît à l'étudiant et que les écritures sont gelées.
11. Étudiant : créer une réclamation et en changer seul le statut.
12. Coupure de réseau : arrêter le serveur, montrer que l'emploi du temps de n'importe quelle semaine de l'année s'affiche toujours depuis le téléphone.

14. Préparer un déploiement hébergé

Rien n'est déployé à ce jour : cette section liste les prérequis, sans modifications faites dans le code. La cible étudiée est Vercel (Projet) avec MongoDB Atlas en base hébergée.

Point	Contrainte réelle	Action obligatoire avant mise en ligne
Base de données	Le code se connecte à `127.0.0.1:27017` et ne connaît pas l'URI Atlas	Créer un cluster, définir `MONGO_URL` et `MONGO_DB`, autoriser l'IP 0.0.0.0/0 ou la plage des adresses des fonctions, ré-exécuter la migration
Secret de session	Repli de démonstration en dur	Définir `ARP_SECRET` aléatoire ; sinon toute session est fabriquable
Comptes	Mot de passe admin de départ connu, jeu de démonstration public	Changer le mot de passe admin, purger les comptes et notes de démonstration
Base locale auto-démarrée	`server-https.js` et `scripts/ensure-mongo.js` lancent `mongod.exe` : impossible hors du poste	Ne pas déployer le serveur custom ; repartir sur `next start` ou les fonctions du serveur
Certificats / mkcert / tunnel	Tout le `scripts/` TLS est propre au réseau local	À écarter du déploiement ; Vercel fournit le HTTPS
Système de fichiers	Aucune écriture disque dans `app/` et `lib/` : tout est en base, pièces jointes comprises	Rien — c'est le point qui rend le déploiement possible
Taille des fichiers	Limite de corps de requête de l'offre gratuite ≈ 4,5 Mo, alors que l'application accepte 8 Mo	Baisser la constante de taille, ou passer par un stockage d'objets
Durée des appels IA	`DELAI_MS = 45 000` dans `lib/gemini.ts` contre 10 secondes maximum en offre gratuite	Soit l'offre payante, soit réduire le délai à environ 8 secondes, soit rendre les appels IA asynchrones
reCAPTCHA	La console Google ne liste que `localhost` ; avec un widget v2, un domaine non déclaré affiche son erreur directement dans la case à cocher	Ajouter chaque hôte réellement servi — l'IP du poste, l'hôte du tunnel, le domaine publié — avant de parler de l'anti-robot en démonstration
Clé Gemini	`.assistant-secret` n'est pas dans le dépôt	Recréer la clé comme secret d'environnement ; ne jamais la commiter
Dépôt public	Le dépôt en ligne est public et contient les identifiants de démonstration	Le passer privé, ou nettoyer l'historique avant exposition
Dépendance native	`better-sqlite3` exige une compilation native	Retirer ce paquet, qui ne sert plus qu'au script de migration, et documenter la migration comme archivée

15. Tests réalisés et résultats

Chaque ligne correspond à une exécution réelle sur la version produite, en HTTPS local sur l'adresse du poste, ou par requêtes authentifiées en curl avec un cookie de session fabriqué localement. Aucun test automatisé n'existe : la qualité est établie par ces vérifications manuelles, reproductibles avec les mêmes commandes.

15.1 Socle

Vérification	Méthode	Résultat
Typage et compilation	`npx tsc --noEmit` puis `npm run build`	Sans erreur ; toutes les routes et pages générées
Redirections après écriture	Codes 303 sur les points d'écriture, enchaînement d'un POST puis d'un rechargement	Conforme — aucun rejouage de POST, aucune erreur 500
Retour au même filtre	POST depuis une liste filtrée, lecture de l'en-tête Location	Conforme — `?retour=` conserve onglet, classe, période et page
Redirections depuis le téléphone	Comparaison de l'en-tête Location entre appel par IP et appel par localhost	Conforme après introduction de `clientOrigin`
Garde sans session	Requête anonyme sur `/notes` et sur `/api/notes`	Page : redirection vers la connexion. API : 401 JSON
Accès directs aux pages réservées	Sessions professeur et étudiant sur `/users`	Redirection vers le tableau de bord avec motif
Refus d'écriture hors rôle	Étudiant créant une notification, une salle ou un compte ; professeur écrivant dans une matière d'un autre	403 sur les cas testés
Confidentialité avant validation	Consultation étudiant sur une matière « terminée »	Conforme — zone « en attente de publication »
Gel après validation	Suppression d'une note et dépôt d'un compte rendu sur matière `valide`	Refusés
Verrou de semaine	Écriture sur une semaine à partir du samedi	Refusée côté serveur ; fenêtre d'édition absente
Clôture puis validation	Saisie de trois notes d'examen, publication, calcul de la moyenne pondérée	Conforme
PDF de l'emploi du temps	GET authentifié, contrôle de `Content-Disposition`, ouverture du fichier	Conforme — A4 paysage, accents corrects, grille complète
PDF falsifié pour une autre classe	Paramètre `classe_id` modifié par un étudiant	Conforme — le fichier renvoyé est celui de sa propre classe
Contexte sécurisé et confiance TLS	`curl --cacert certs/arp-root-ca.pem https://<IPv4>/login`	200 sans avertissement ; code 60 lorsque le certificat ne couvre plus l'IP courante

15.2 Cours, RAG et QCM

Vérification	Méthode	Résultat
Import de 25 chapitres de démonstration	`npm run cours:gen` puis `npm run db:cours`, c'est-à-dire l'API réelle avec session, droits et extraction	Conforme — les 25 chapitres créés, textes extraits, indexés
Extraction .docx	Fichiers générés par python-docx, lecture par le parseur maison	Conforme

Extraction .pdf	Décompression des flux puis décodage par les tables ToUnicode	Conforme sur les PDF produits ; le repli de police sans CMap donne du texte partiel, d'où le seuil de 300 caractères
Indexation sémantique	Nombre de passages avec vecteur non vide après import	Conforme — passages indexés en 768 dimensions, normalisés
Recherche cosinus	Question posée sur un chapitre précis, lecture des passages retenus	Conforme — les 4 passages retenus proviennent du bon chapitre
Repli sans vecteur	Requête lorsqu'un passage n'a pas de vecteur	Conforme — bascule sur le recouvrement de mots, affichée à l'écran
Une seule tentative	Deux démarrer sur la même clé, puis double soumission	Refusé par l'index unique ; la note ne change pas
Corrigé non exposé	Lecture de la réponse de la route GET avant soumission, dans le code source de la page	Conforme — aucune bonne réponse dans le HTML envoyé
Note séparée	Passation d'un QCM puis lecture de la moyenne de matière	Conforme — le quiz n'altère aucune moyenne
Protection du chapitre	Suppression et réouverture après passations	Refusées

15.3 Assistant

Vérification	Méthode	Résultat
Chemin IA réel	Questions libres des trois rôles, lecture du champ `source`	Conforme — `source:"ia"`, réponses de 950 à 1 250 caractères, trois suggestions, quota décrémenté
Refus des cinq familles	Questions types : modifier mes notes, donner un mot de passe, données d'un autre élève	Conforme — refus avant tout appel réseau au modèle
Étanchéité	Question d'un étudiant sur un autre étudiant	Refus ; et le snapshot ne contient de toute façon pas la donnée
Repli sans clé	Variable de clé rendue vide, puis questions	Conforme — réponses locales, `assistant_usage` non incrémenté
Repli sur clé invalide	Fausse clé temporaire en place du modèle réel	Conforme — erreur 400, modèle suivant, puis repli local ; clé supprimée après le test
Quota	Appels au-delà de la limite journalière	Conforme — bascule en local pour le reste de la journée
Droits sur la route	Appel d'API avec un rôle auquel `peut()` refuse le module	403 — la matrice reste la seule source de droits même sans page dédiée

15.4 Comptes rendus avec pièce jointe

Vérification	Méthode	Résultat
Dépôt avec .docx	POST multipart réel, puis relecture du dépôt	Conforme — binaire stocké, nom, type et taille conservés, texte pré-rempli
Extraction avant stockage	Route d'extraction isolée, sans écriture en base	Conforme — le texte s'affiche avant validation, rien n'est écrit
Fichier de 7 Mo	Chronométrage de l'extraction	0,76 s — pas de risque d'occupation du processeur
Fichier trop lourd	Envoi au-delà de 8 Mo	Refusé par un message explicite *avant* tout traitement du contenu
Format non pris en charge	.txt puis fichier aux deux en-têtes absentés	Refusés (415) ; l'extension déclarée ne suffit pas

Pièce jointe scannée	PDF sans couche texte	Acceptée et téléchargeable ; le texte reste à saisir à la main
Remplacement	Modification de dépôt sans nouveau fichier	Conforme — l'ancienne pièce jointe est conservée
Contrôle d'accès au fichier	Téléchargement par son auteur, par le professeur responsable, par un autre étudiant, par un autre professeur	Les deux premiers autorisé, les deux autres refusés
Nom de fichier malveillant	Nom contenant guillemets et retours chariot	Nettoyé avant l'en-tête de téléchargement
Gel de la note	Modification de dépôt après attribution de note	Refusée, comme prévu par la règle métier

15.5 Anti-robot

Vérification	Méthode	Résultat
Module désactivé sans les deux clés	Variables laissées vides, lecture de la page de connexion	Conforme — aucun script Google chargé, formulaire strictement classique
Branchement côté serveur	Appel direct de la fonction de vérification avec un jeton vide	Conforme — refus motivé, motif écrit dans la sortie du serveur et non dans la page
Markup v2 servi	Lecture de la page produite après build, avec les clés chargées	Conforme — script `api.js` chargé **sans** `?render=` , un seul conteneur `div.g-recaptcha` porteur du `data-sitekey` du fichier `.env.local` , aucun champ `g-recaptcha-response` dans le HTML serveur (il est créé par Google au clic)
Widget réellement peint dans le navigateur	Arbre accessible de la page `/login` après chargement	Conforme — l'iframe `reCAPTCHA` apparaît dans le formulaire, avant le champ email, sans erreur dans la console
POST sans jeton, en conditions réelles	Requête envoyée au port 443 sans aucune valeur `g-recaptcha-response`	Conforme — 303 vers `/login?captcha=1` , et le motif « aucun jeton » écrit dans la sortie du serveur ; le mot de passe n'est pas même testé
Jetons périmés ou fabriqués	Le jeton v2 est à usage unique et expire au bout de deux minutes ; le contrôle d'`origin` l'attache à la clé du site	Conforme en lecture — un jeton d'un autre domaine est refusé par `siteverify` . Non testé avec un vrai jeton volé, faute de second site
Clic sur la case puis connexion	Test dans le navigateur intégré	**Non concluant par nature :** l'onglet piloté est masqué, et reCAPTCHA ne laisse pas cocher sa case sur une page non visible. La validation finale doit être faite dans une vraie fenêtre Chrome, sur un domaine déclaré dans la console Google

15.6 Défauts rencontrés pendant le développement et leur correction

Défaut	Origine	Correction
Erreur 500 après chaque connexion et chaque formulaire	Redirection 307 rejouant le POST sur une page serveur	Redirection 303 systématique, centralisée dans `routeurRetour()`
Bouton d'impression cassant l'écran	Gestionnaire d'événement passé dans un composant serveur	Bouton isolé dans un composant client
« Télécharger en PDF » ouvrait la fenêtre d'impression	Aucune bibliothèque de génération de PDF prévue	Vrai fichier PDF produit par le serveur, sans dépendance ajoutée
Le téléphone renvoyé sur une adresse injoignable	Origine issue de Next et non de la demande réelle	`lib/origin.ts` utilisé par toutes les redirections

Route ajoutée introuvable ou suspendue après un build	Le serveur Node sert la sortie de build figée au dernier `npm run build`	Rebuild obligatoire avec toute modification de route ; procédure écrite dans la documentation
Port 443 refusé au redémarrage	Le processus lancé par le wrapper npm survivait à l'arrêt de ce wrapper	Vérification de la ligne de commande du processus occupant, puis arrêt dirigé ; le serveur est relancé directement par Node
Clé d'environnement semblant ignorée	Variable déclarée vide dans le shell : elle écrase la valeur du fichier `.env.local`	Relancer le serveur avec l'environnement propre ; le diagnostic a d'abord été cherché dans le code, à tort
Widget « Type de clé non valide » sur la page de connexion	Version de clé et version de code incompatibles : le type choisi dans la console Google (v2 ou v3) ne correspondait pas au markup servi. Le message est affiché au propriétaire du site, pas au visiteur, et il n'existe qu'en combinaison v3	Diagnostic mené par la lecture de la console Google plutôt que par le code : la page montre le type réel de la clé. La clé a été recréée en v2 et le module laissé en v2 ; les deux modes restent documentés séparément
Aucun jeton produit en v3 alors que le script était chargé	Le composant s'exécutait au montage, avant l'arrivée d' `api.js` , et reCAPTCHA v3 ne génère rien dans un onglet masqué	Attendu avec l'apparition d'une clé v3. Ce mode ayant été abandonné au profit de la case à cocher, le problème ne se pose plus : c'est Google qui remplit le champ du formulaire au clic, sans intervention du code de l'application
Réponses de l'assistant coupées en plein milieu	Tokens de « réflexion » remplissant la sortie maximale	`thinking_budget: 0` et sortie portée à 1 600 tokens
Assistant silencieusement sans IA	Identifiants de modèles codés en dur, devenus indisponibles (404) ; le repli les masquait	Liste de modèles réellement disponibles, ordonnée, avec bascule sur erreur ; la source est affichée à l'utilisateur
Haut de la conversation inaccessible	`scrollIntoView` inefficace dans un parent en position fixe, comportement fluide suspendu quand l'onglet est masqué	Positionnement direct de la zone par `scrollTop`

16. Limites connues et évolutions possibles

16.1 Limites assumées

- **Courriels simulés.** Le module consigne les messages dans la collection `emails` avec leur lien : aucun serveur SMTP n'est contacté, ce qui permet une démonstration hors ligne et sans boîte réelle. Le passage à l'envoi véritable consiste à remplacer `lib/mail.ts` par un transporteur SMTP en gardant la même signature. Le lien de l'e-mail de démonstration contient un jeton d'exemple : il est volontairement faux et mène à « lien invalide ».
- **Version de Next contrainte.** Le maintien en Node 18.13 fixe Next en 13.5 : pas d'actions serveur, dépendances figées.
- **Trace de SQLite conservée.** `better-sqlite3` reste dépendance du projet et `data/arp.db` existe encore : ni l'un ni l'autre ne sont utilisés au runtime, uniquement par le script de migration. À supprimer au premier déploiement.
- **Base mono-poste.** La connexion est mémoïsée par processus et mongod est lancé par l'application : une installation à deux serveurs exigerait une base hébergée.
- **Aucune projection en couche de lecture.** `findDocs` retourne les documents entiers, donc les binaires : sur les listes de comptes rendus et de chapitres, les pièces jointes sont relues avec les documents. Payant mais sans effet observable aux volumes de démonstration ; la correction passe par une option de projection dans `lib/db.ts`.
- **Repli hors ligne limité à l'emploi du temps** (volontaire, cf. section 12.4), sans file d'attente de modifications.
- **Installabilité Android liée au certificat.** Chrome peut refuser une autorité installée par l'utilisateur : le tunnel public devient alors nécessaire.
- **Aucun test automatisé** : ni tests unitaires, ni scénarios de bout en bout rejouables. La section 15 est un protocole manuel, pas une suite.
- **Pas d'expiration temporelle des jetons** d'invitation, pas de limitation du nombre de tentatives de connexion en dehors de l'anti-robot, pas de journal d'audit des accès.
- **Similarité cosinus calculée en JavaScript** : inadapté à des milliers de passages ; documenté comme tel plutôt que présenté comme du vector search.
- **Certains libellés ne suivent pas** : le bandeau « réservé à l'administration » s'affiche pour tout refus d'accès, y compris hors admin ; l'encart de démonstration de la page de connexion peut promettre un mot de passe qui ne marche plus ; le pied de page du tableau de bord contient une entité HTML visible dans une chaîne de caractère.
- **Fonction morte :** `resumeDroits()` est exportée sans être appelée ; aucun écran n'affiche les droits en toutes lettres.

16.2 Évolutions naturelles

1. Envoi réel des courriels avec modèle HTML et expiration temporelle des jetons.
2. Déploiement hébergé (section 14) : Atlas, secrets d'environnement, domaine publié.
3. File d'attente hors ligne des pointages, rejouée à la reconnexion, avec accusé visuel.
4. Tests automatisés : unités sur `lib/` (semaine, moyennes, extraction, matrice) et scénarios de bout en bout par rôle.
5. Stockage des pièces jointes hors base, ou au moins projection en lecture pour ne pas charger les binaires.
6. Règle d'examen : signalement automatique au-delà d'un nombre d'absences ou de retards.
7. Bulletin imprimable par étudiant, avec les styles d'impression de l'emploi du temps.

8. Historique des versions de chapitre, et non plus un texte remplacé à chaque envoi.
9. Refonte de l'anti-robot en limitation de débit côté serveur, indépendant de Google et valable aussi hors connexion Internet.

17. Annexe — arborescence

Chemin	Contenu
`app/`	18 écrans (composants serveur) + `app/api/` 22 points d'entrée + mise en page racine et styles globaux
`components/`	13 composants, dont 9 marqués client : Sidebar, Modal, ConfirmForm, Filtres, Pagination, Stats, EmploiTableau, HorsLigneSemaines, AssistantChat, AssistantFlottant, DepotJoint, PrintButton, PwaRegister
`lib/`	18 modules : db, permissions, auth, filtres, retour, semaine, origin, cours, rag, qcm, gemini, assistant, assistant-ia, assistant-contexte, hors-ligne, recaptcha, pdf, mail
`middleware.ts`	Garde des pages et API ; chemins publics : `/login`, `/reset`, `/offline`, `/api/auth/login`, `/api/auth/reset`. Ne vérifie que la forme du cookie : les droits vivent dans la matrice
`public/`	`manifest.webmanifest`, `sw.js` (cache `arp-v6`), `icons/icon.png`
`scripts/`	ensure-mongo.js (lanceur de base), ensure-cert.js (TLS auto-adaptatif), migrate-sqlite-to-mongo.js, show-collections.js, import-cours.js, gen-docx.py, start-mongod.bat, make-cert.ps1, get-mkcert.ps1, expose-phone.sh
`server-https.js`	Exposeur HTTPS de la version produite, port 443, lance mongod, signe et applique le certificat
`data/`	Reste de la base SQLite d'origine ; hors suivi de versions, plus utilisé au runtime
`certs/`	Certificat, **clé privée** et autorité publique mkcert ; hors suivi de versions, ne doit jamais être transmis
`.assistant-secret`, `.tunnel-secret`, `.env.local`	Secrets locaux : clé Gemini, secret de session, clés reCAPTCHA. Hors suivi de versions
`docs/`	Source HTML de ce document, à conserver dans le dépôt pour qu'il ne soit plus perdu
`README.md`, `README-ANDROID.md`	Démarrage général ; procédure détaillée téléphone

Document généré le 7 novembre 2026 pour la version 3.1 d'ARP Éducation — Next.js 13.5.11, MongoDB 8.0.10 (driver 7.7.0), TypeScript 5.9.3, Tailwind 3.4.19, Node 18.13, reCAPTCHA v2, API Gemini.